TECHNICAL TRAINING · OPTIMIZE AI MODELS

Shrinking AI models

One weight stored in 32 bits



The same weight stored in 8 bits



START HERE · READ IN ORDER

What we will cover

1	**The problem**	Why big models need smaller numbers
2	**Numbers in a computer**	Bits, bytes and the formats that store weights
3	**The big idea**	What quantization is, and how scale and zero point work
4	**Your choices**	Symmetric or asymmetric, and how many scales to use
5	**Real models**	Building a quantizer, what it saves, and sharing it
6	**Going lower**	Packing 2-bit and 4-bit weights, and smarter methods
7	**Putting it together**	How an engineer decides, a self-check and a glossary

PART 1 · THE PROBLEM

Big models, small hardware

A modern language model can have **7 billion** weights, which are the numbers it learned during training. Stored at full precision, that is about **28 GB**, far more than the memory of most laptops. We cannot even load it, let alone run it.



Quantization is how we close this gap.

PART 2 · NUMBERS IN A COMPUTER

Bits and bytes

A **bit** is the smallest piece of information a computer stores: a 0 or a 1. Every extra bit **doubles** the number of different values you can write down.

2 bits
4
different values

4 bits
16
different values

8 bits
256
different values

16 bits
65,536
different values

32 bits
4.3
billion different values

8 bits make 1 byte, the unit behind KB, MB and GB. More bits give finer detail, but every single number then takes more memory.

PART 2 · NUMBERS IN A COMPUTER

How a decimal is stored

A weight is usually a decimal like **0.03452**. A computer stores it the way scientists write numbers: a sign, a power of two, and the significant digits. These are the real 32 bits of that number.



Sign · 1 bit

0 means positive, 1 means negative.

Exponent · 8 bits

The power of 2 that sets the size. Here it works out to −5.

Mantissa · 23 bits

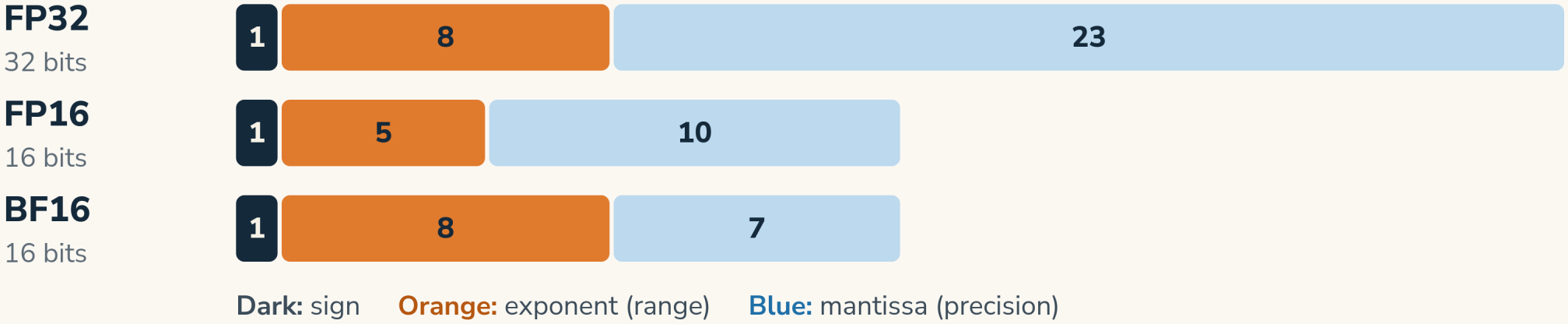
The digits of the number. Here they work out to 1.10464.

So 0.03452 ≈ 1.10464 ÷ 32. The 32 bits are a recipe for rebuilding the number, not the number itself.

PART 2 · NUMBERS IN A COMPUTER

FP16 and BF16

Both use 16 bits, half of FP32. They split those bits differently. **Exponent bits set the range** (how big or small a number can be). **Mantissa bits set the precision** (how much detail it keeps).



FP16: finer detail, small range

Its largest number is 65,504, only 5 digits long.

BF16: rougher detail, huge range

Same range as FP32. Its largest number has 39 digits.

BF stands for Brain Floating Point. It came from Google Brain.

PART 2 · NUMBERS IN A COMPUTER

FP16 and BF16 in practice

Here is what each format actually stores when we hand it the same number. Look for the two places where FP16 breaks.

Original number	FP16 stores	BF16 stores
0.03452	0.034515	0.034424
1000.7	1000.5	1000
70,000	Too big: overflow	70,144
0.00000001	Too small: becomes 0	0.00000001

FP16 is more precise

Its answers land closer to the original, until a number gets too big or too small.

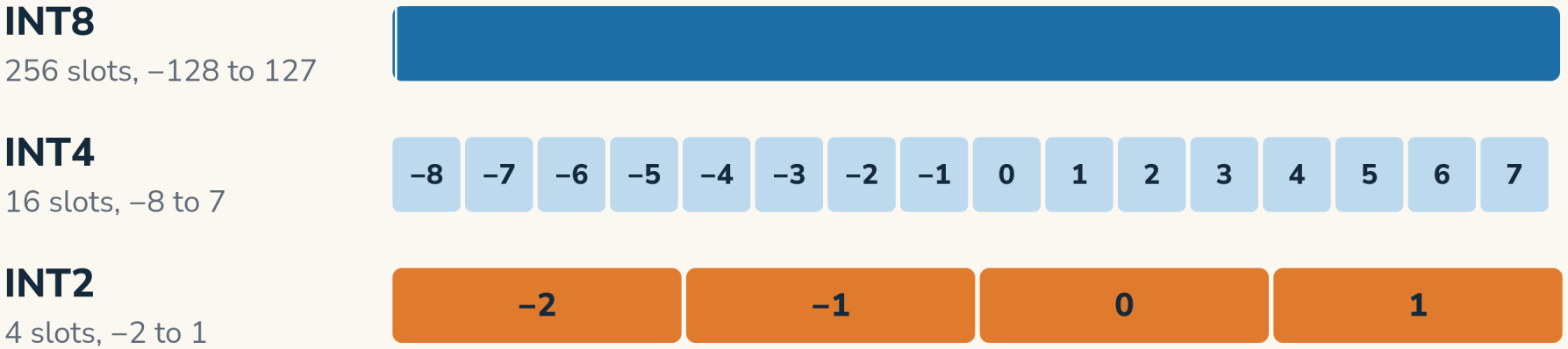
BF16 rarely overflows

It copes with extreme values like FP32, which is why modern models like it.

PART 2 · NUMBERS IN A COMPUTER

Whole-number formats

Integers have no decimal part, only whole numbers in a fixed range. That is exactly what makes them small. **INT8** keeps 256 slots, **INT4** keeps 16, and **INT2** keeps just 4.

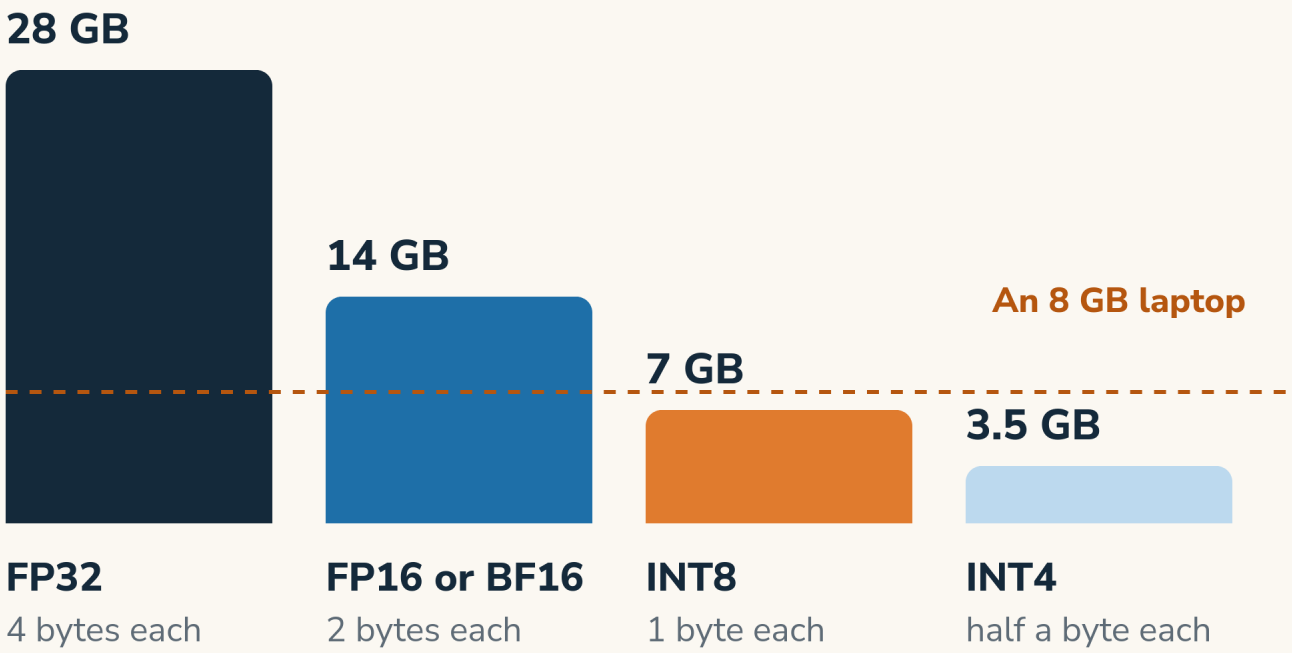


FP32 can tell apart about 4.3 billion values, INT8 only 256. **Fewer slots means every stored number is rougher.** Quantization is the craft of using those few slots wisely.

PART 2 · NUMBERS IN A COMPUTER

From bits to model size

A model with 7 billion weights



THE SIMPLE RULE

Size = number of weights × bytes per weight

Same model, same number of weights. Only the storage format changed, and the size fell from 28 GB to 3.5 GB.

The easiest first win

Load a model in BF16 instead of FP32. It halves the size with almost no loss in quality. This is called downcasting. Quantization goes further.

PART 3 · THE BIG IDEA

What quantization is

Quantization means storing numbers with fewer bits. We squeeze a huge range of decimals into a small set of whole numbers, and we remember how to translate them back.

BEFORE · FP32 · 4 BYTES EACH

0.03452 · −1.57 · 0.8812



AFTER · INT8 · 1 BYTE EACH

3 · −127 · 71

plus one shared scale: 0.01236

To read them back, multiply each whole number by the scale: 0.0371 · −1.57 · 0.8777. Close to the originals, not identical.

An everyday picture: rounding 23.47 birr to 23 birr. You drop the santim, the number is easier to store, and you can still work with it. Quantization does the same to weights, with one extra step. We first stretch or shrink the values so they use the whole range of slots. That stretch factor is the **scale**.

PART 3 · THE BIG IDEA

The trade-off

What we gain

Smaller. A 7-billion-weight model drops from 14 GB to 7 GB at 8 bits.

Faster. Less data to move around, and simpler math on whole numbers.

Wider reach. Models can run on laptops, phones and small devices.

What we pay

Some accuracy. Rounding creates a small error, called the quantization error.

More squeeze, more error. Going from 8 bits to 4 or 2 costs more each time.

Uneven damage. Some layers and some models are more sensitive than others.

In this training we quantize models that are **already trained**. This is called post-training quantization, and it needs no retraining.

PART 3 · THE BIG IDEA

Scale and zero point

THE WHOLE RECIPE IN ONE LINE

real value = scale × (whole number − zero point)

To store a value we run it backwards: divide by the scale, add the zero point, then round to the nearest whole number.

Scale

How much real value one whole-number step is worth. A big scale means coarse steps. A small scale means fine steps.

Zero point

Which whole number stands for the real number 0. It shifts the range so that zero is stored exactly.

One scale and one zero point are shared by many weights, so their cost is tiny.

PART 3 · THE BIG IDEA

Finding scale and zero point

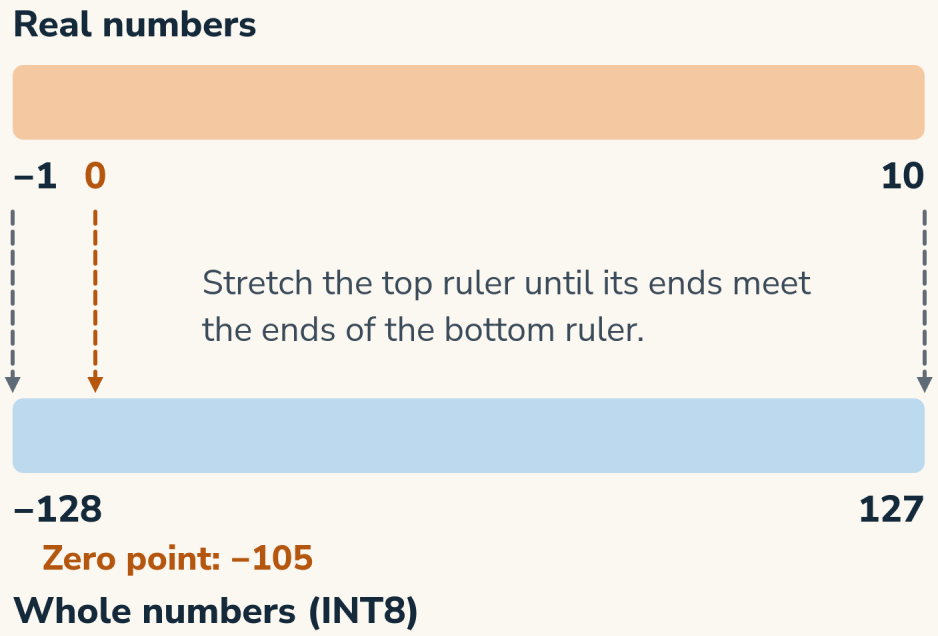
1. **Find the real range**
   Smallest −1, largest 10 in our example.

2. **Find the whole-number range**
   INT8 allows −128 to 127, which is 255 steps.

3. **Scale = real spread ÷ whole-number spread**
   11 ÷ 255 ≈ 0.0431 per step.

4. **Zero point = where real 0 lands, rounded**
   Real 0 sits about 23 steps above −128, so it lands near −105.

The zero point is a whole number, so real 0 is stored **exactly**, with no error.



PART 3 · THE BIG IDEA

A worked example

Four values, stored in INT8 with scale **0.0431** and zero point **−105**.

Original	Stored	Read back	Gap
−1	−128	−0.992	0.008
0	−105	0	0
2	−59	1.984	0.016
10	127	10.008	0.008

Follow the value 2

1. Divide by the scale: 2 ÷ 0.0431 = 46.4

2. Add the zero point: 46.4 − 105 = −58.6

3. Round it: **−59**. This is what we store.

4. Read it back: scale × (−59 + 105) = scale × 46 ≈ 1.984

Every value comes back within **0.016** of the original, and real 0 comes back exactly. Four bytes shrank to one.

PART 4 · YOUR CHOICES

Symmetric and asymmetric

These are two ways to lay values onto the whole-number slots. We use the same data as before, real values from −1 to 10, in 8 bits. The bars show all 256 slots.

Symmetric
scale 0.0787, no zero point



Asymmetric
scale 0.0431, zero point −105



Symmetric mode must reach out to ±10 to cover the 10, so most of the negative side sits empty. Asymmetric mode fits the data exactly.

1.8×
finer steps in asymmetric mode here

PART 4 · YOUR CHOICES

Choosing a mode

	Symmetric	Asymmetric
Range	−max to +max, centered on zero	Real minimum to real maximum
Zero point	Always 0, nothing to store	Needed, stored with each scale
Effort	Simpler, faster math	A little more work
Slots	Wasted if data is lopsided	All used
Best when	Data is balanced around zero	Data is lopsided (never negative)

Rule of thumb from the course: symmetric is typical at 8 bits. Asymmetric is often preferred at 2 to 4 bits, where every slot counts.

PART 4 · YOUR CHOICES

The outlier problem

One scale shared by every number has a weakness. Take six small weights, then add one loud value, 12.0. Both cases use 8 bits and a single scale.

Original	0.20	−0.10	0.30	0.05	0.15	−0.25
Without 12.0 scale 0.0024	0.2008	−0.0992	0.3000	0.0496	0.1512	−0.2504
With 12.0 scale 0.0945	0.1890	−0.0945	0.2835	0.0945	0.1890	−0.2835

40×

coarser steps, because the loud value set the scale for everyone.

The quiet values get squeezed

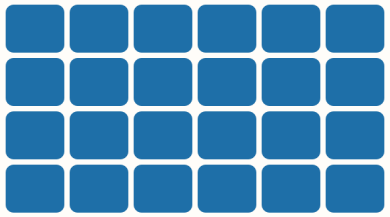
The six small weights are now stored as just 2, −1, 3, 1, 2 and −3. The weight 0.05 comes back as 0.0945, almost double.

PART 4 · YOUR CHOICES

Three levels of granularity

Granularity means how many numbers share one scale. In each picture below, cells of the same color share a scale.

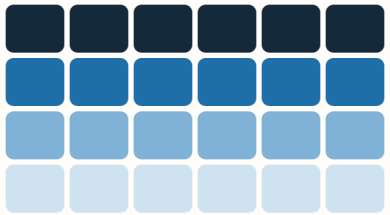
Per tensor



Here: 1 scale

One scale for the whole table. Simple, but one outlier affects everything.

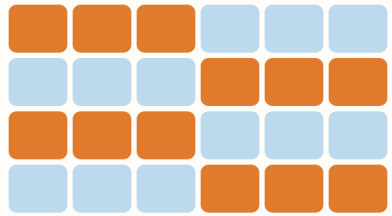
Per channel



Here: 4 scales

One scale for each row or column. An outlier only hurts its own row.

Per group



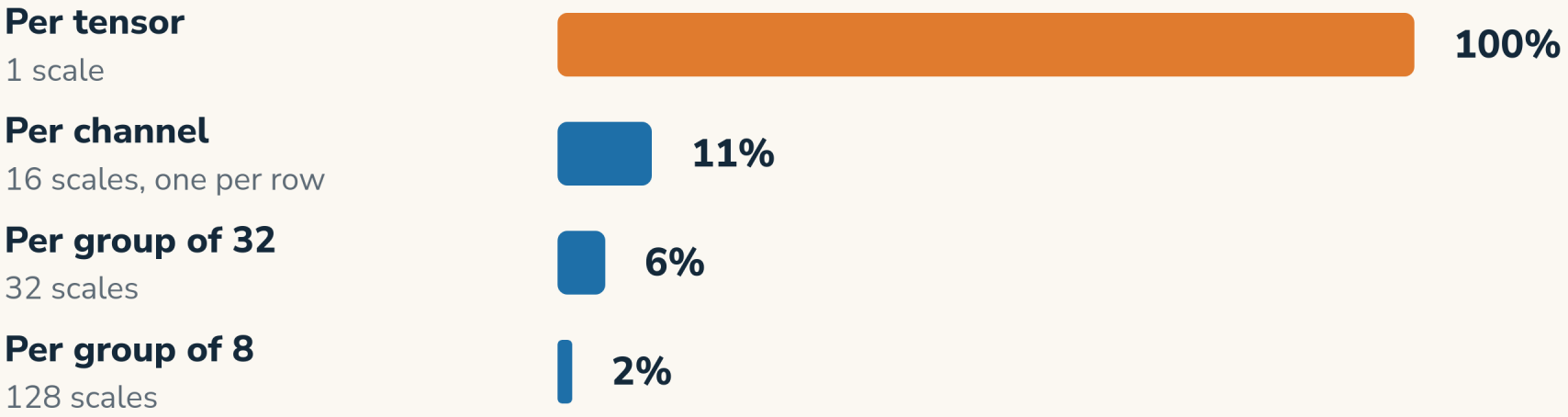
Here: 8 scales

One scale for each small block of values, often 32, 64 or 128. The finest control.

PART 4 · YOUR CHOICES

The price of precision

Quantization error, compared with one scale for everything (lower is better)



An illustration on a small made-up table of weights (16 rows of 64 values, with two outliers), 8 bits, symmetric.

The catch

Every scale is a number that must be stored too. With 4-bit weights and a 16-bit scale for every 32 weights, each weight really costs 4 + 16 ÷ 32 = **4.5 bits**.

A common choice

At 8 bits, one scale per channel is the usual pick. Its extra storage is tiny, and an outlier only hurts its own channel.

PART 4 · YOUR CHOICES

Measuring the error

How do we know a setup is good enough? We compare what we stored with what we started with, then we look at the real model.

1

Pick a setup

Choose the bits, the mode and the granularity.

2

Measure the gap

Average the difference between original and read-back numbers. Lower is better.

3

Check the model

Look at its size, and test whether real outputs still look right.

4

Compare, decide

Choose the option that best meets your constraint.

One number is not the whole story. A small average gap can hide a bad result on the cases that matter, so always test real outputs too. Engineers work in this loop: try, measure, compare, justify.

PART 5 · REAL MODELS

Weights and activations

Weights

The learned numbers stored in the model. They are fixed once training is done.

Activations

The values flowing from layer to layer while the model runs. They change with every input.

So what do we quantize?

Weights only

Store small weights, then convert them back for the math. Saves memory and works on any hardware.

Weights and activations

Do the math itself in whole numbers. Can be faster, but not all hardware supports it.

W8A16 means 8-bit Weights and 16-bit Activations. This is the setup the course builds.

PART 5 · REAL MODELS

Building a quantizer

1. Start with a trained model in FP16 or BF16.
2. Walk through its layers and find the **dense (linear) layers**, where most weights live.
3. Quantize each one's weights: 8 bits, symmetric, **one scale per output channel**.
4. Swap in a **quantized layer** that stores the 8-bit weights and their scales.
5. Keep sensitive layers, like the final output layer, at **full precision**.

One quantizer, many models

It only looks at layers and numbers, so it works on text, vision and audio models alike.

For using, not training

The quantized weights are fixed. This is made for running a model.

PART 5 · REAL MODELS

Inside a quantized layer

What it stores

The weights, as 8-bit whole numbers.

One scale for each output channel.

A bias, if the original layer had one.

What it does each time the model runs

1. Convert the stored 8-bit weights to the working precision, FP16 or BF16.
2. Do the usual multiplication with the incoming activations.
3. Multiply the result by the scales. One cheap correction at the end, not one per weight.
4. Add the bias, if there is one.

The weights **sit in memory in 8 bits**, but the math runs in normal precision. That is why quality stays close to the original.

PART 5 · REAL MODELS

Layers we leave alone

WHY

A language model writes one piece at a time, and each output becomes the next input. A tiny rounding error in the final output layer gets repeated and can grow until the text turns into gibberish. So that layer stays at full precision.

CodeGen, a code-writing model

About 350 million weights. After 8-bit quantization it still wrote correct code for a simple task.

DETR, an object detector

It still found the same objects in a test photo. Its box and label layers stayed unquantized.

A warning for bigger models. Rounding errors matter more as models grow, roughly above 6 billion weights, as the course instructor notes. That is why smarter methods exist, and we meet them later.

PART 5 · REAL MODELS

What we really save

In theory, 8 bits is half the size of 16 bits. In practice some parts of a model stay at higher precision, so the real saving is smaller. These numbers come from the course lessons.

250 → 166 MB

OPT-125M, a small language model

Half-precision file against the quantized file, about one third smaller.

25–30%

DETR, an object detector

About 50 MB saved from about 170 MB. Its convolution layers were not quantized.

The more of a model's weights sit in the layers you quantize, the more you save. Always measure the real file size, not the theory.

PART 5 · REAL MODELS

Share once, load anywhere

Loading a full model just to quantize it needs a lot of memory. So we do the heavy work once, on a big machine, and share the result.

ON A MACHINE WITH PLENTY OF MEMORY

1. Load the full model
2. Quantize it
3. Save the quantized weights
4. Upload to the Hugging Face Hub



ON YOUR LAPTOP

1. Download the quantized weights
2. Make an empty model skeleton
3. Swap in the quantized layers
4. Fill it with the weights and use it

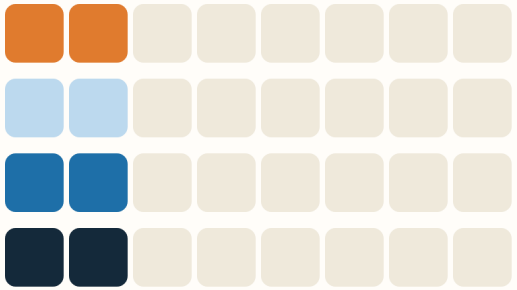
The full-precision model is never loaded on the small machine. That means smaller downloads and less memory.

PART 6 · GOING LOWER

The packing problem

Below 8 bits there is a catch. The smallest storage box in a computer is 8 bits wide. A 2-bit number in an 8-bit box leaves 6 bits empty, so 4-bit or 2-bit weights save nothing on their own.

Without packing



4 bytes for 4 weights. Most of each box is empty.

With packing

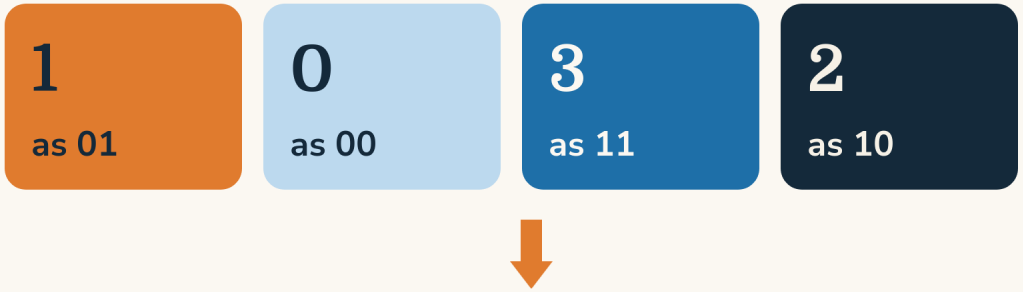


1 byte for the same 4 weights. Four times less memory.

PART 6 · GOING LOWER

Packing step by step

Four 2-bit weights: 1, 0, 3, 2



One 8-bit box



10110001 read as a whole number is 177

The first weight sits at the **right end**. Each next weight goes two places to the left.

To use the weights, **unpack** them: slide each pair back into place and read it.

The count must fill whole boxes: a multiple of 4 for 2-bit weights, of 2 for 4-bit.

The trade-off: less memory, a little extra work at run time.

PART 6 · GOING LOWER

Beyond linear quantization

Big language models grow a few enormous values, outliers, just like the loud value we met earlier. Plain linear quantization struggles with them, so researchers built smarter methods.

LLM.int8

Split the work. The few outlier values are computed in higher precision, everything else in 8 bits, and the two results are combined.

SmoothQuant

Share the difficulty. It rescales so the activations' outliers move partly into the weights, making both easier to quantize to 8 bits.

AWQ

Look at real data first. A small calibration dataset reveals the weights that matter most, and those are protected before quantizing.

Also worth knowing: GPTQ quantizes layer by layer and corrects rounding errors as it goes. QLoRA fine-tunes a model that is kept in 4 bits.

PART 7 · PUTTING IT TOGETHER

How an engineer decides

1

What is the constraint?

Memory, speed or a quality target. Write it as a number.

2

How many bits?

Start at 8. Go lower only if you must.

3

Which mode?

Is the data balanced around zero, or lopsided?

4

How many scales?

Tensor, channel or group. Watch outliers and storage.

5

Which layers stay full?

Start with the final output layer.

6

Did it work?

Measure size, error and real outputs, then compare options.

There is no single right answer. **A good solution is one you can justify with measurements.**

PART 7 · PUTTING IT TOGETHER

Key takeaways

1. A model is billions of numbers. Fewer bits per number means a smaller, faster model.
2. FP16 keeps detail, BF16 keeps range, and integers like INT8 keep only whole numbers.
3. Linear quantization needs a scale and a zero point, then rounding.
4. Symmetric is simpler, asymmetric uses every slot. Finer granularity is more accurate but costs storage.
5. Leave sensitive layers alone, and always measure real results.
6. Below 8 bits, pack values into bytes. For big models, use smarter methods against outliers.

PART 7 · PUTTING IT TOGETHER

Check yourself

Try these on your own before turning the page. The answers are on the next slide.

1. A model has 3 billion weights stored in FP16. How big is it? How big in INT8?

2. Which has the larger range, FP16 or BF16? Which has the finer detail?

3. What do the scale and the zero point each do?

4. Data runs from −1 to 10. Which mode wastes more slots, and why?

5. Why can one outlier ruin a quantization that uses a single scale?

6. Why do we pack 2-bit weights into 8-bit boxes?

PART 7 · PUTTING IT TOGETHER

Answers

1 **FP16:** 3 billion × 2 bytes = 6 GB. **INT8:** 3 billion × 1 byte = 3 GB.

2 **BF16** has the larger range, the same as FP32. **FP16** has the finer detail.

3 The **scale** is the real value of one whole-number step. The **zero point** is the whole number that stands for real 0.

4 **Symmetric.** It must reach out to ±10, so about 45% of the slots go unused.

5 The biggest value sets the scale. A loud outlier makes every step coarse, so small values collapse onto a few whole numbers.

6 The smallest storage box is 8 bits. Packing fits four 2-bit weights into one box, so memory use matches the real bit width.

PART 7 · PUTTING IT TOGETHER

Glossary

Weight	One learned number inside a model.
Bit and byte	A bit is a 0 or a 1. A byte is 8 bits.
FP32, FP16, BF16	Decimal formats using 32, 16 and 16 bits.
INT8, INT4	Whole-number formats using 8 and 4 bits.
Quantization	Storing numbers with fewer bits.
Scale	The real value of one whole-number step.
Zero point	The whole number that stands for real 0.
Symmetric	Range centered on zero, no zero point.
Asymmetric	Range fitted to the data, with a zero point.
Granularity	How many numbers share one scale.
Outlier	A value much larger than the rest.
Activation	A value passed between layers while a model runs.
Inference	Using a trained model to produce answers.
Packing	Fitting several small numbers into one byte.